

Chapter 4 — K-Nearest Neighbors (KNN)

Aymen Negadi

July 2026

Unlike Logistic Regression and Decision Trees, K-Nearest Neighbors (KNN) is a completely different type of algorithm. It does not build a mathematical model during training. Instead, it stores the training data and makes predictions only when a new observation needs to be classified. Because of this, KNN is called a **lazy learning algorithm** or **instance-based learning algorithm**.

1 What is K-Nearest Neighbors (KNN)?

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for both classification and regression. In this chapter, we focus on classification.

The main idea is very intuitive: *Objects that are close to each other are likely to belong to the same class.*

When a new data point arrives, the algorithm:

1. Computes the distance between the new observation and every training sample.
2. Selects the K nearest neighbors.
3. Counts the class labels of those neighbors.
4. Assigns the class with the majority vote.

Unlike Logistic Regression, KNN does not estimate probabilities using the sigmoid function. Unlike Decision Trees, KNN does not ask a sequence of decision rules. Instead, it relies entirely on the similarity between observations.

2 Why is it Called K-Nearest Neighbors?

The name has two parts:

- **K:** The letter K represents the number of neighbors considered when making a prediction (e.g., $K = 1$, $K = 3$, $K = 5$, $K = 7$).
- **Nearest Neighbors:** The neighbors are the training observations that are closest to the new observation according to a distance metric.

Therefore, K-Nearest Neighbors means: *“Look at the K closest observations and let them vote.”*

3 Why Do We Use KNN?

Imagine you move into a new neighborhood and want to know whether the area is quiet. Instead of reading complicated reports, you ask the five closest neighbors. If four of them say the neighborhood is quiet and one says it is noisy, you would probably conclude that the neighborhood is quiet.

KNN works exactly the same way: it trusts the labels of the nearest observations.

4 Real-World Applications

KNN is widely used in many fields:

- **Healthcare:** Disease diagnosis, medical image classification.
- **Finance:** Credit risk prediction, fraud detection.
- **Computer Vision:** Face recognition, handwritten digit recognition.
- **Recommendation Systems:** Movie recommendation, product recommendation.
- **Text Classification:** Spam detection, document classification.

5 How Does KNN Work?

Suppose we have the following training dataset:

Age	Salary	Purchased
22	30,000	No
25	35,000	No
30	50,000	Yes
35	60,000	Yes
40	70,000	Yes

Table 1: Training Dataset

Now a new customer arrives with Age = 32 and Salary = 55,000. The goal is to predict whether this customer will purchase the product.

The algorithm performs the following steps:

- **Step 1:** Store all training observations.

- **Step 2:** Measure the distance between the new customer and every training observation.
- **Step 3:** Select the K nearest neighbors.
- **Step 4:** Count the class labels.

For $K = 3$:

Neighbor	Class
1	Yes
2	Yes
3	No

Votes: Yes = 2, No = 1 → **Prediction:** Yes.

6 Choosing the Value of K

The value of K is one of the most important hyperparameters:

- **Small K (e.g., $K = 1$):**
 - *Advantages:* Very simple, captures local patterns.
 - *Disadvantages:* Sensitive to noise, can overfit.
- **Large K (e.g., $K = 15$):**
 - *Advantages:* More stable, less sensitive to noise.
 - *Disadvantages:* May ignore local structure, can underfit.

Choosing an appropriate value of K is usually done using validation techniques.

7 Distance Metrics

KNN depends entirely on measuring the distance between observations. The most common distance metrics are Euclidean Distance, Manhattan Distance, and Minkowski Distance.

The Euclidean Distance is the most widely used. For two points $A(x_1, y_1)$ and $B(x_2, y_2)$, the Euclidean distance is:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \quad (1)$$

In higher dimensions (n features):

$$d = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2)$$

8 Worked Example 1

Suppose the training data are:

Point	Coordinates	Class
A	(1,2)	Blue
B	(2,3)	Blue
C	(4,5)	Red
D	(5,6)	Red

A new point is $Q = (2, 2)$.

Distances:

- $d(Q, A) = 1$
- $d(Q, B) = 1$
- $d(Q, C) \approx 3.61$
- $d(Q, D) = 5$

For $K = 3$, the neighbors are A (Blue), B (Blue), and C (Red). **Votes:** Blue = 2, Red = 1 $\rightarrow$ **Prediction:** Blue.

9 Worked Example 2 — 2D Points Detail

Using 2D points is the standard way to explain KNN because students can easily visualize the neighbors.

Suppose we have the following training dataset:

Point	Coordinates	Class
A	(1, 1)	Blue
B	(2, 2)	Blue
C	(3, 1)	Blue
D	(6, 5)	Red
E	(7, 6)	Red

Now suppose a new point arrives $Q = (3, 3)$. Our goal is to determine its class using the K-Nearest Neighbors algorithm.

9.1 Step 1 — Choose the Value of K

Let $K = 3$.

9.2 Step 2 — Compute the Euclidean Distance

Compute the distance from $Q = (3, 3)$ to every training point:

- **Distance to A:**

$$d(Q, A) = \sqrt{(3-1)^2 + (3-1)^2} = \sqrt{4+4} = \sqrt{8} \approx 2.83$$

- **Distance to B:**

$$d(Q, B) = \sqrt{(3-2)^2 + (3-2)^2} = \sqrt{1+1} = \sqrt{2} \approx 1.41$$

- **Distance to C:**

$$d(Q, C) = \sqrt{(3-3)^2 + (3-1)^2} = \sqrt{0+4} = \sqrt{4} = 2.00$$

- **Distance to D:**

$$d(Q, D) = \sqrt{(6-3)^2 + (5-3)^2} = \sqrt{9+4} = \sqrt{13} \approx 3.61$$

- **Distance to E:**

$$d(Q, E) = \sqrt{(7-3)^2 + (6-3)^2} = \sqrt{16+9} = \sqrt{25} = 5.00$$

9.3 Step 3 — Sort the Distances

Point	Distance	Class
B	1.41	Blue
C	2.00	Blue
A	2.83	Blue
D	3.61	Red
E	5.00	Red

9.4 Step 4 — Select the K Nearest Neighbors

Since $K = 3$, the nearest neighbors are B (Blue), C (Blue), and A (Blue).

9.5 Step 5 — Majority Voting

Votes: Blue = 3, Red = 0.

Final Prediction: The algorithm predicts that $Q = (3, 3)$ belongs to the **Blue** class because all of its three nearest neighbors are Blue.

10 Advantages and Limitations

10.1 Advantages

- Very easy to understand and implement.
- No explicit training phase.
- Works well on small datasets.
- Naturally handles multiclass classification.
- Can model complex decision boundaries.

10.2 Limitations

- Slow prediction phase on large datasets.
- Requires storing the entire training dataset.
- Sensitive to irrelevant features and feature scaling.
- Performance depends heavily on the choice of K and the distance metric.

11 Important Observation

Unlike other algorithms:

- **Logistic Regression** learns weights and predicts probabilities.
- **Decision Trees** learn decision rules.
- **KNN** learns no mathematical model; it simply memorizes the training data and compares new observations with stored examples.